INFORME TÉCNICO: IMPLEMENTACIÓN MULTIMOTOR Y ADAPTACIÓN NoSQL (BIG DATA / IoT)

Proyecto: Red Social Estudiantil Pascualina

Fase: Implementación Física, Multimotor y Datos Semiestructurados

Nombre: Andrés Mauricio López Gil

Grupo 1, estoy trabajando solo.

FASE DE DESARROLLO

1. Propuesta de creación e integración NoSQL (IoT / Big Data)

Se seleccionó **PostgreSQL** como el Sistema Gestor de Base de Datos (SGBD) principal para la implementación debido a su soporte nativo superior para datos semiestructurados mediante el tipo `JSONB`.

Caso de Uso de IoT y Big Data implementado: Para aplicar los conceptos de *Big Data e IoT*, se adaptó la tabla `asistente_evento`. En el campus de la Institución Universitaria Pascual Bravo, la asistencia a eventos institucionales se registrará mediante **tarjetas inteligentes (RFID) y nodos de conexión Wi-Fi (IoT)**. Cada vez que un estudiante ingresa a un evento, los sensores envían una carga útil (payload) con datos heterogéneos: dirección MAC del dispositivo, ID del sensor de la puerta, intensidad de señal y temperatura del nodo. Al ser datos variables que cambian según el tipo de sensor, se agregó el campo `registro_iot` de tipo **JSON**, permitiendo almacenar esta información semiestructurada. Esto generará un volumen masivo de datos (Big Data) que a futuro permitirá analizar mapas de calor, patrones de movilidad en el campus y aforos en tiempo real.

2. Espacio de retroalimentación grupal

Durante la simulación de despliegue en tres motores (PostgreSQL, MySQL, SQL Server), se recibieron las siguientes observaciones del equipo y docente:

- **Sobre la sintaxis de Autoincremento:** Se observó que el SGBD principal (PostgreSQL) usa `SERIAL` o `IDENTITY`, mientras que MySQL exige `AUTO_INCREMENT` y SQL Server `IDENTITY(1,1)`.
- **Sobre el tipo de dato JSON:** Se resaltó que aunque PostgreSQL y MySQL tienen un tipo `JSON` nativo (y PostgreSQL tiene el optimizado `JSONB`), SQL Server tradicionalmente almacena JSON en campos `NVARCHAR(MAX)` validados con la función `ISJSON()`.
- **Decisión:** Se acordó construir un **Diccionario de Datos Físico comparativo** para estandarizar la migración a cualquiera de los tres motores sin perder la integridad del diseño.

3. Memoria justificativa y ajuste final

El modelo final mantiene la rigidez relacional (3FN) para datos transaccionales críticos (usuarios, grupos, eventos), pero flexibiliza sus bordes (Edge Computing) al adoptar enfoques NoSQL dentro de bases relacionales. Utilizar `JSONB` en PostgreSQL para la telemetría IoT (`registro_iot`) y para los metadatos de las publicaciones (`metadatos_adjuntos`) asegura que la base de datos pueda escalar y absorber la variabilidad del Big Data sin necesidad de rediseñar las tablas cada vez que se agregue un nuevo sensor en el campus.